

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
FACULTAD DE INGENIERÍA

Profesor(a):	Ariel Adara Mercado Martínez
Asignatura:	Estructuras de Datos y Algoritmos I
Grupo:	5
No. de Práctica(s):	Proyecto Final
Integrante(s):	De Jesús Carranza Giovanni Gabriel Enriquez Bahena Fernando Israel González Mateo Carlos
Nombre de equipo:	Tosti
Semestre:	2026-2
Fecha de entrega:	11 de junio de 2026

CALIFICACIÓN: _____

Índice

1. Introducción	3
2. Estructuras de Datos Implementadas	3
2.1. Pila Dinámica (Stack).....	3
2.2. Cola Dinámica (Queue) y Cola Circular.....	3
2.3. Administración de Memoria Dinámica.....	3
3. Módulo Analizador (Parser) y Procesamiento de Archivos	4
3.1. Lectura y Mapeo de Variables (cargarArchivo).....	4
3.2. Filtrado de Datos.....	4
4. Algoritmo de Planificación y Asignación de Recursos	4
4.1. Políticas de Despacho y Quantum.....	4
4.2. Ejecución y Ciclo de Vida del Proceso.....	4
5. Casos de Prueba y Resultados	5
6. Dificultades encontradas	5
7. Conclusiones	6

1. Introducción

Este proyecto aborda el diseño, la simulación y la evaluación del comportamiento dinámico de un núcleo de Sistema Operativo simplificado (*Mini-OS*). En un entorno de cómputo real, la coexistencia de múltiples procesos que demandan recursos finitos de procesamiento (CPU) y almacenamiento voluntario (RAM) exige una capa de abstracción de software intermedia que controle el acceso de manera equitativa y eficiente.

Las arquitecturas clásicas de planificación y asignación lineal directa sufren de problemas severos de monopolización y fragmentación de recursos. Para resolver esta situación de forma óptima sin las limitantes de los arreglos estáticos tradicionales, se implementó un sistema modular cooperativo en lenguaje C acoplado a scripts analíticos en Python 3. El simulador gestiona la multiprogramación a través de un planificador a largo plazo enlazado y un despachador Round Robin que interactúa concurrentemente con un administrador de memoria RAM dinámica regido por la estrategia *First-Fit*.

2. Estructuras de Datos Implementadas

Todas las colecciones de datos dentro del micronúcleo fueron desarrolladas desde cero utilizando abstracciones dinámicas de memoria para mitigar el desbordamiento de estructuras estáticas.

2.1. Pila Dinámica (Stack)

La pila opera bajo el estricto principio LIFO (*Last In, First Out*), donde el último elemento en ser ingresado a la cima es el primero en ser desalojado. En el sistema actual, se implementó mediante nodos enlazados donde cada elemento conserva un valor entero y un apuntador de autorreferencia al nodo inferior. Sus funciones primitivas (*stack push*, *stack pop* y *stack is empty*) sirven como soporte de control para flujos secuenciales auxiliares e inversión de estados dentro del micronúcleo.

2.2. Cola Dinámica (Queue) y Cola Circular

El sistema de planificación requiere dos variantes específicas del paradigma FIFO (*First In, First Out*), donde el primer elemento en ingresar es el primero en ser despachado:

- **Cola Lineal Dinámica:** Mantiene referencias explícitas al frente (*front*) y al fondo (*rear*) mediante nodos enlazados simples. Se utiliza como la *Job Queue* (Cola de Trabajos) que simula el almacenamiento secundario donde residen los procesos en espera cuando la memoria RAM se encuentra saturada.
- **Cola Circular:** Implementada sobre un arreglo con indexación basada en aritmética modular. Actúa como la *Ready Queue* (Cola de Listos) para el planificador de la CPU. Permite la inserción y extracción de identificadores en tiempo constante $O(1)$ sin desplazamientos de memoria físicos.

2.3. Administración de Memoria Dinámica

A través de peticiones explícitas gestionadas con la función *malloc()*, el sistema inicializa una estructura lineal basada en una lista doblemente ligada que representa la memoria RAM libre y asignada. Cada bloque físico (*MemoryBlock*) registra la dirección de inicio, el tamaño del segmento y el estado de ocupación. Al finalizar el ciclo de cálculo de cada proceso, se ejecutan de manera asíncrona procedimientos con la instrucción *free()* para liberar y desasignar los nodos del *heap*, previniendo pérdidas de recursos del sistema (*memory leaks*).

3. Módulo Analizador (Parser) y Procesamiento de Archivos

El módulo analizador actúa como el puente de datos interlenguaje, transformando los archivos planos de texto generados por los scripts de análisis en colecciones de estructuras legibles por el kernel.

3.1. Lectura y Mapeo de Variables (cargarArchivo)

El simulador implementa una rutina estructurada de lectura utilizando apuntadores de flujo FILE*. El archivo de entrada processes.csv se procesa línea por línea de forma controlada mediante la función fgets(). El parser filtra y descarta la primera línea correspondiente a la cabecera del archivo. Posteriormente, mediante la instrucción estricta sscanf, extrae de forma limpia las variables enteras correspondientes al Identificador del Proceso (PID), Tiempo de Ráfaga (Burst Time), Prioridad y Memoria Requerida, mapeándolas directamente a la tabla global de atributos.

3.2. Filtrado de Datos

Durante el ciclo de análisis del archivo CSV, el parser ejecuta subrutinas de validación sintáctica. Se omiten de forma automática caracteres extraños, espacios en blanco y saltos de línea (\n y \r\n) a través de delimitadores controlados. Los registros conformados por operandos válidos se encapsulan en estructuras nativas C y se introducen de manera secuencial a la cola de trabajos, protegiendo al simulador maestro contra corrupciones de memoria ante archivos corruptos.

4. Algoritmo de Planificación y Asignación de Recursos

El núcleo unificado entrelaza las operaciones de la memoria RAM y la CPU mediante máquinas de estado concurrentes.

4.1. Políticas de Despacho y Quantum

El planificador a corto plazo aplica la política *Round Robin* sobre los procesos residentes en la memoria RAM. Para garantizar equidad, se estableció un intervalo de tiempo fijo denominado *Quantum* ($q = 2$). Cuando un proceso toma el control de la CPU, consume unidades de reloj equivalentes al Quantum. Si su tiempo remanente es mayor a cero, es desalojado y reencolado al fondo de la *Ready Queue* circular, evitando la inanición (*starvation*) del sistema.

4.2. Ejecución y Ciclo de Vida del Proceso

El ciclo maestro opera bajo las siguientes restricciones algorítmicas:

1. **Asignación First-Fit:** Se recorre secuencialmente la lista doble de memoria. El primer bloque libre que cumpla con el tamaño requerido se asigna al proceso. Si el bloque es de mayor tamaño, se ejecuta la función splitBlock(), fragmentando el nodo de forma exacta.
2. **Coalescencia Asíncrona:** Al finalizar la ráfaga de CPU de un proceso, el bloque de memoria RAM asignado cambia su estado a libre. Inmediatamente se invoca la subrutina coalesce memory(), unificando bloques libres contiguos de manera lineal en una sola pasada con complejidad espacial $O(1)$.

5. Casos de Prueba y Resultados

Las pruebas de integración y rendimiento se ejecutaron compilando el código modular nativo a través de la herramienta gcc dentro de un entorno Linux (Ubuntu/WSL). Se generó un lote estocástico inicial de $n = 40$ procesos a través de Python.

```
=== INICIANDO SIMULADOR MINI-OS (ENTORNO N-PROCESOS) ===
Carga exitosa: 40 procesos en la Cola de Trabajos (Disco).
--- FASE 2: EXECUCIN DE SIMULACIN DINMICA ---
[RELOJ: 0 u] -> P1 entro a la RAM (Direccion base: 0, Tamao: 244 MB).
[RELOJ: 0 u] -> P2 entro a la RAM (Direccion base: 244, Tamao: 82 MB).
[RELOJ: 0 u] -> P3 entro a la RAM (Direccion base: 326, Tamao: 186 MB).
[RELOJ: 0 u] -> P4 entro a la RAM (Direccion base: 512, Tamao: 205 MB).
[RELOJ: 0 u] -> P5 entro a la RAM (Direccion base: 717, Tamao: 146 MB).
[RELOJ: 1 u] -> P1 FINALIZO. RAM liberada en direccion 0.
[RELOJ: 1 u] -> P6 entro a la RAM (Direccion base: 0, Tamao: 151 MB).
[RELOJ: 51 u] -> P6 FINALIZO. RAM liberada en direccion 0.
[RELOJ: 51 u] -> P7 entro a la RAM (Direccion base: 0, Tamao: 236 MB).
[RELOJ: 53 u] -> P2 FINALIZO. RAM liberada en direccion 244.
[RELOJ: 55 u] -> P3 FINALIZO. RAM liberada en direccion 326.
[RELOJ: 57 u] -> P4 FINALIZO. RAM liberada en direccion 512.
...
[RELOJ: 316 u] -> P40 FINALIZO. RAM liberada en direccion 673.
[RELOJ: 317 u] -> P39 FINALIZO. RAM liberada en direccion 608.
[RELOJ: 320 u] -> P37 FINALIZO. RAM liberada en direccion 136.
[C] Resultados consolidados en 'data/outputs/resultados.csv'
```

Listing 1: Log de ejecución exitosa.

Los archivos de salida se cruzaron de manera automatizada a través del script graphs.py, exportando a la carpeta reports/png/ cuatro gráficas técnicas de control que demuestran la consistencia matemática de la simulación.

6. Dificultades encontradas

Durante el desarrollo e integración modular del ecosistema surgieron diversas anomalías técnicas que requirieron auditorías manuales y correcciones de código:

Errores de compilación por directivas de comentarios: Al integrar las primeras notas de diseño dentro del archivo principal src/main.c, se utilizaron de forma instintiva caracteres de almohadilla (#), propios de lenguajes como Python. Esto causó fallos críticos en el compilador de C, que interpretó el texto como directivas corruptas del preprocesador. La dificultad se solventó reemplazándolos por comentarios de tipo lineal estandarizados (//).

Colisiones de alcance de punto de entrada (Multiple Definition of Main): La

automatización de compilación implementada por el Makefile para la instrucción `make test` incluía indiscriminadamente todos los archivos `.c`. Al coexistir el main de pruebas unitarias y el main del simulador, el enlazador (`ld`) generó un fallo destructivo. La solución consistió en reestructurar el Makefile utilizando la función `$(filter-out)` para aislar las rutinas.

- **Fallos de E/S por directorios ausentes en el Sistema de Archivos:** Durante el cierre de la simulación, la instrucción `fopen()` con permisos de escritura fallaba de manera persistente al intentar volcar el archivo `resultados.csv`. Se detectó que el gestor de versiones Git ignora las carpetas vacías por diseño, omitiendo la existencia física de la ruta `data/outputs/`. Se inyectaron comandos preventivos `mkdir -p` para asegurar el entorno.

7. Conclusiones

La realización de este proyecto integrador consolidó los conceptos teóricos abstractos de la asignatura administrándolos en un escenario de ingeniería de software real y funcional. La implementación de colas lineales, estructuras circulares modulares y listas doblemente enlazadas permitió comprender a fondo cómo los Sistemas Operativos organizan y mitigan los cuellos de botella lógicos en la administración del procesador y el almacenamiento.

Asimismo, se reforzó el dominio sobre la gestión manual del mapa de memoria dinámica en lenguaje C, adquiriendo experiencia crítica en la prevención de fallos de segmentación (*Segmentation Faults*) mediante la validación rigurosa de apuntadores de referencia antes y después de transformaciones estructurales. El acoplamiento interlenguaje demostró la relevancia de combinar lenguajes eficientes de bajo nivel (C) para el núcleo de cómputo con herramientas ágiles de alto nivel (Python) para la analítica avanzada.